INDUSTRY INTERNSHIP REPORT

On

WEB DEVELOPMENT

By

Paramapogu Kathrin Hepsiba
(229X1A05F0)

Submitted in accordance with the requirement for the degree of B-Tech

Department of Computer Science and Engineering

G. PULLA REDDY ENGINEERING COLLEGE (Autonomous): KURNOOL

(Affiliated to JNTUA, ANANTHAPURAMU)

KURNOOL - 518007

2025 – 2026

Department of Computer Science and Engineering

G. PULLA REDDY ENGINEERING COLLEGE (Autonomous): KURNOOL

(Affiliated to JNTUA, ANANTHAPURAMU)

CERTIFICATE

This is to certify that the internship titled 'Web development' is a Bonafide record of work carried out by Paramapogu Kathrin Hepsiba (229X1A05F0), and the Internship report is submitted in partial fulfillment of requirements for the award of degree of Bachelor of Technology in Computer Science and Engineering.

Internship Guide
Dr. K. V. Rameswar Reddy
Professor,
Department of C.S.E.,
G. Pulla Reddy Engineering College,
Kurnool.

Head Of the Department
Dr. N. Kasiviswanath
Professor & Head of the Department,
Department of C.S.E.,
G. Pulla Reddy Engineering College,
Kurnool.

INTERNSHIP DETAILS

Name of the College: G. Pulla Reddy Engineering College

Department: Computer Science and Engineering

Name of the Student: Paramapogu Kathrin Hepsiba

Register Number: 229X1A05F0

Name of the Guide: Ms. Susmithaa Sasmal

Duration of the Internship: 12 weeks **From:** 15-12-2025 **To:** 15-03-2026

Name and address of the Organization: Plasmid Innovations Pvt. Ltd., 1st floor, Featherlight The Address, Marathahalli, Bangalore, KA, 560103.

Internship Title: Web Development

InternShip Stipend: NA

Date of Internship Report Submission:

Internship Coordinator

Dr. B. Geetha Vani

Professor

CSE Department

GPREC

DECLARATION

I**, Paramapogu Kathrin Hepsiba**, a student bearing the Roll No. **229X1A05F0,** of the Department of Computer Science and Engineering, G. Pulla Reddy Engineering College, do hereby declare that the I have completed the industry internship titled Web Development from 15th December, 2025 to 15th March 2026 in, Plasmid Innovations Pvt. ltd, 1st floor, Featherlight The Address, Marathahalli, Bangalore, KA, 560103, under the Guideship of Ms. Susmithaa Sasmal, Director of Organization at Plasmid Innovations Pvt. Ltd, Bangalore, who provides guidance in specialized training programs such as Web Development.

(Student Signature and Date)

CERTIFICATE FROM ORGANIZATION

This is to certify that P.K.Hepsiba , Reg No: 229X1A05F0 of G. Pulla Reddy Engineering College, Kurnool, undervent an Internship in Plasmid Innovation Pvt. Ltd, from 15-12-2025 to 15-03-2026.The overall performance of the Intern during his/her internship is found to be Good.

INTRAINZ INNOVATION PRIVATE LIMITED

Signature and Date

Internship completion certificate

PLASMID

MSME
MICRO, SMALL & MEDIUM ENTERPRISES
Development & Facilitation Office

Certificate of Internship

Is proudly awarded to

Paramapogu Kathrin Hepsiba

This is to certify that Ms. Paramapogu Kathrin Hepsiba has successfully completed an internship with Plasmid in the domain of Web Development from December 15th 2025 to March 15th 2026. Her performance in the program was commendable and her efforts were found to be sincere and diligent

K Praveen Kumar,
Senior Manager,
Operations & HR

ID: 1225CWB10004

ACKNOWLEDGEMENT

I wish to express my deep sense of gratitude to my internal supervisor **Dr. B. GeethaVani** Professor of Computer Science and Engineering Department, G. Pulla Reddy Engineering College.

My sincere thanks to our Internship Coordinator **DR. B. GEETHAVANI** for her immaculate guidance, constant encouragement and cooperation which have made possible to bring out this internship work.

My sincere thanks to **Ms**. **Susmithaa Sasmal**, Director of Organization, for helping me and giving me the required information needed for my internship work.

I am thankful to our Head of the Department **DR. N. KASIVISWANATH**, for his whole hearted support and encouragement during the internship.

We are grateful to our Internship Dean **DR. Y. V. SIVA REDDY** and to our respected Principal **DR. B. SREENIVASA REDDY** for providing requisite facilities and helping us in providing such a good environment.

I wish to convey my acknowledgements to all the staff members of the Computer Science and Engineering Department for giving the required information needed for my internship work.

Finally, I wish to thank all our friends and well-wishers who have helped me directly or indirectly during the course of this internship work.

Table of Contents

Chapter 1: **INTRODUCTION** 1

1.1 About the Internship Company/Organization 2

1.2 Virtual/Offline Internship details 3

Chapter 2: **WORK DONE DURING INTERNSHIP** 5

2.1 About Course/Domain 6

2.2 Technologies learnt during internship 6

Chapter 3: **PROJECTS/MODULES COMPLETED** 27

3.1 Web Applications 28

3.2 Concepts Demonstrated 32

3.3 About Project 1 / Module 1 32

3.4 About Project 2 / Module 2. 32

Chapter 4: **ACTIVITY LOG** 48

Chapter 5: **CONCLUSION** 53

Evaluation Report……

References 54

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
2.1	Document Object Model	10
2.2	Spring Boot Project Developement Life	17
2.3	Rest API Development Workflow	24
2.4	Angular Application Architecture	26
2.5	Layered Onion Architecture of Web Application	30
2.6	Agile Scrum Framework for project management	31
3.1	Input from user and calculating the monthly EMI	34
3.2	Bargraph representation of the EMI	34
3.3	Pictorial representation of monthly EMI	37
3.4	Amortization table	37
3.5	Listing of Employees	44
3.6	Listing of Department	45
3.7	Adding of Department	45
3.8	Updating the department table	46

CHAPTER 1:
INTRODUCTION

CHAPTER 1: INTRODUCTION

1.1 About the Internship Organization

The internship was carried out at Plasmid Innovations, a technology-focused training organization dedicated to bridging the gap between academic learning and industry requirements in the domain of web development. Plasmid's mission is to equip students with industry-ready skills by combining structured theoretical instruction with practical, hands-on implementation. Their programs are designed to follow a modern, progressive curriculum that gradually builds competencies from foundational concepts to advanced full-stack development.

The training environment at Plasmid emphasizes interactive and collaborative learning. Students are guided through live sessions, project-based assignments, and real-world scenarios that reinforce each concept as it is introduced. This incremental approach ensures that learners not only understand the theory but also gain confidence in applying it. The curriculum begins with the basics of how the internet works, then progresses through HTML, CSS, JavaScript, Bootstrap, and Saas, before extending into back-end development with Spring and database management using SQL. This structured pathway ensures that students develop a holistic understanding of both front-end and back-end technologies.

One of the key strengths of Plasmid's training program is its emphasis on practical application. Every module includes exercises, coding challenges, and mini-projects that encourage students to implement what they have learned. This methodology fosters technical proficiency, logical thinking, and problem-solving ability, while also instilling the confidence to work independently on real-world projects. By the end of the program, students are not only familiar with the tools and frameworks but also capable of designing and deploying functional applications.

The internship undertaken in this environment culminated in the development of a fully functional BMI Calculator an interactive front-end web application built using HTML, CSS, and JavaScript. This project served as a capstone, integrating all the front-end skills acquired during the training period. It demonstrated the effectiveness of Plasmid's approach, where theoretical knowledge was consistently reinforced through practical implementation, ultimately preparing students to meet industry expectations with confidence.

Additional Information:

Official Page: https://plasmid.co.in/

Course Page (Internship Domains): https://intrainz.mykademy.com/login

LinkedIn: https://www.linkedin.com/company/plasmid23/

Login Page: https://intrainz.mykademy.com/login

Real-World Project Exposure:

Students are immersed in the end-to-end software development lifecycle, including internships that replicate a hybrid onsite-offshore development model. This gives learners a feel of professional environments, deadlines, and teamwork, preparing them thoroughly for their careers ahead.

LEARNING OVERVIEW DURING THE INTERNSHIP:

During my internship at Plasmid Innovation Pvt. ltd, I underwent a structured and practical learning journey that focused on developing both technical and professional skills. The training was designed in a way that combined theoretical understanding with hands-on implementation, enabling me to gain real-world experience in full-stack development.

Approach to Learning:

The internship followed a disciplined daily schedule that included learning sessions, hands- on practice, and presentations. Each concept was first explained by mentors, followed by implementation through coding exercises and mini-projects. This approach helped me understand concepts deeply and apply them effectively in real-world scenarios.

Daily presentations played a crucial role in my learning process. By explaining concepts regularly, I was able to improve my communication skills, strengthen my understanding, and gain confidence in presenting technical topics.

Practical Implementation:

A major part of my learning involved working on real-world projects. These projects helped me understand the complete software development lifecycle, including planning, development, testing, and deployment.

Through these projects, I also learned how to:

- Debug and fix errors efficiently
- Write clean and maintainable code
- Collaborate with team members
- Manage time and meet deadlines

1.2 Offline Internship Details

The Offline Internship Program at Plasmid Innovation Pvt. Ltd. is a comprehensive, 12-week, full-time internship designed to transform students into confident, industry-ready developers. This in-person training experience offers immersive learning through real-world projects, expert mentorship, and daily performance tracking.

Plasmid's training environment is equipped with structured live sessions, hands-on project assignments, and guided mentorship. These facilities support a wide range of learning activities, allowing students to experiment, innovate, and develop cutting-edge solutions. The organization also promotes a collaborative and interactive learning environment where students actively engage in discussions, assignments, and technical projects, thereby creating opportunities for knowledge exchange and exposure to current industry trends.

The Department of Web Development at Plasmid is particularly known for its focus on emerging technologies and modern development methodologies. The curriculum covers fundamental and advanced topics. It also provides opportunities for students to engage in hands-on projects, assessments, and real-time development activities, thereby enhancing their technical expertise and practical understanding.

Duration & Timings

- Total Duration: 12 Weeks
- Timings: Monday to Saturday, from 9:00 AM to 5:00 PM
- Location: Plasmid Innovation pvt. Ltd. 1st floor, Featherlight, Marathahalli, Bangalore.

Program Structure

This offline internship is crafted to mirror the dynamic environment of software development companies. The daily schedule is structured to build technical expertise, encourage communication, and promote self-driven learning.

Daily Learning + Practice

Interns engage in hands-on training sessions where they code, debug, and develop real-world applications under the guidance of experienced mentors.

Daily Presentations (2 Hours)

A dedicated 2-hour session each day is reserved for interns to present their learnings and progress. This encourages:

- Consistent revision and knowledge retention
- Improved communication and presentation skills
- Confidence in articulating technical concepts

Mentorship & Guidance

The faculty at Plasmid Innovation Pvt. Ltd consists of highly experienced mentors, many from prestigious institutions (IITs, IISc) and top tech companies (Google, Microsoft, Meta). Their teaching style is approachable, encouraging students to ask questions and actively participate.

Tech Stack & Tools

Interns work on modern technologies and industry tools including:

- **Programming:** Java, JavaScript
- **Frontend:** HTML, CSS, React.js
- **Backend:** Spring Core, Spring Boot
- **Databases:** SQL
- **Tools:** Git, GitHub

Learning Outcomes

By the end of the 1st week, interns will:

- Gain solid experience in building web applications
- Understand the end-to-end software development lifecycle
- Improve collaboration through team-based projects
- Be capable of explaining and documenting their work professionally
- Receive constructive feedback on their presentations and code

Certificate & Recognition

Upon successful completion, interns receive a certificate of internship, which validates their practical experience and can be added to their resume, portfolio, or LinkedIn profile.

This offline internship at Plasmid Innovation Pvt. Ltd is not just about learning how to code, it's about developing the confidence, discipline, and communication skills needed to thrive in the software industry.

CHAPTER 2:
WORK DONE DURING INTERNSHIP

2. WORK DONE DURING INTERNSHIP

2.1 ABOUT COURSE/DOMAIN

Introduction:

During my internship, I gained hands-on experience in **Web Development**, covering both **front-end technologies** (HTML, CSS, JavaScript) and **full-stack development** (React, Spring Boot, MySQL). These projects helped me understand how to design user interfaces, implement business logic, and manage persistent data.

Web Development is a comprehensive domain that involves building interactive, scalable, and user-friendly applications. It spans:

- **Client-side development (Frontend):** Designing user interfaces and ensuring responsiveness.
- **Server-side programming (Backend):** Implementing business logic, APIs, and security.
- **Database management:** Storing, retrieving, and organizing structured data.

Through my projects — **EMI Calculator** and **Employee Management System** — I experienced the full lifecycle of application development, from **UI design** to **backend integration** and **database persistence**.

2.2 TECHNOLOGIES LEARNT DURING INTERNSHIP

1. HTML (Hypertext Markup Language)

HTML was the foundation of my **EMI Calculator project**. It provided the structure for input fields, buttons, and output sections.

Key Learnings:

- Creating structured web pages using semantic tags like <header>, <section>, <article>, and <footer>.
- Designing forms with input fields, labels, buttons, and validation attributes.
- Embedding multimedia elements such as images and charts.
- Organizing content using lists and tables.

Example (Loan Input Form):

```
<form>
  Loan Amount: <input type="number" id="loanAmount" />
  Interest Rate: <input type="number" id="interestRate" />
```

```
  Tenure (Years): <input type="number" id="tenure" />

  <button type="submit">Calculate EMI</button>
</form>
```

2. CSS (Cascading Style Sheets)

CSS was used to style both projects. In the **EMI Calculator**, CSS ensured clarity and usability. In the **Employee Management System**, CSS combined with React components created a professional UI.

Key Learnings:

- Applying styles using inline, internal, and external CSS.
- Using selectors, classes, and IDs to target elements.
- Implementing layouts using **Flexbox** and **Grid**.
- Creating responsive designs using **media queries**.
- Adding animations and transitions for better user experience.

Example (Centering Calculator):

```
.calculator {
  display: flex;
  flex-direction: column;
  align-items: center;
  margin: 20px;
}
```

3. JavaScript

JavaScript powered the **EMI Calculator logic** and dynamic updates.

Key Learnings:

- DOM manipulation to dynamically update content.
- Event handling such as clicks, form submissions, and slider inputs.
- Form validation to ensure correct data entry.
- Functions, loops, and conditional statements for logic.
- Handling asynchronous operations using callbacks and promises.

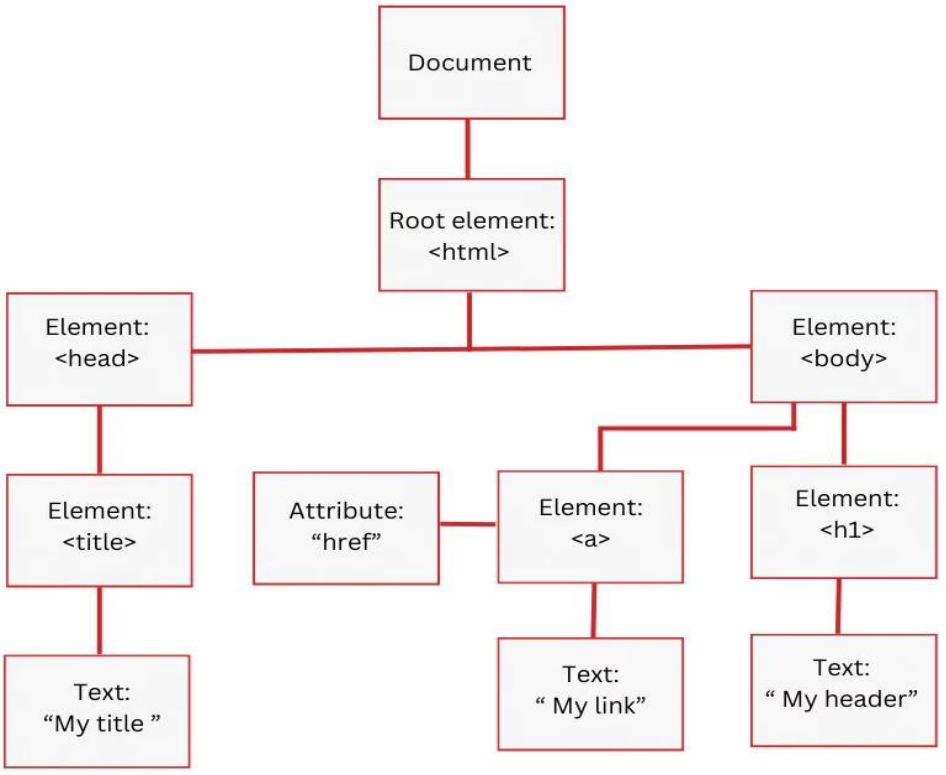


Figure 2.1 Document Object Model (DOM) Tree Structure

Example (EMI Calculation Logic):

```
function calculateEMI(principal, rate, tenure) {
  let monthlyRate = rate / (12 * 100);
 let emi = (principal * monthlyRate * Math.pow(1 + monthlyRate, tenure)) /
            (Math.pow(1 + monthlyRate, tenure) - 1);
  return emi.toFixed(2);
}
```

4. Core Java & Advanced Java

Java was the backbone of the **Employee Management System backend**.

Key Learnings:

- Core Java concepts: OOP (classes, objects, inheritance, polymorphism).
- Advanced Java: JDBC, Servlets, and integration with **Spring Boot**.
- REST API development with **Spring MVC**.
- DTOs and Mappers for clean data transfer.

Example (Spring Boot REST Controller):

```
@RestController
@RequestMapping("/api/employees")
public class EmployeeController {

    @Autowired
    private EmployeeService service;

    @GetMapping
    public List<Employee> getAllEmployees() {
        return service.getEmployees();
    }

    @PostMapping
    public Employee addEmployee(@RequestBody Employee emp) {
        return service.saveEmployee(emp);
    }
}
```

5. **React (Frontend for Employee Management System)**

React was used to build a dynamic, component-driven frontend.

Key Learnings:

- Functional components with **useState** and **useEffect** hooks.
- Client-side navigation using **React Router**.
- Dynamic forms with real-time validation.
- Controlled components for managing user input.

Example (Employee Form Component):

```
function EmployeeForm({ onSubmit }) {
  const [name, setName] = useState("");
  const [dept, setDept] = useState("");

  const handleSubmit = (e) => {
    e.preventDefault();
    onSubmit({ name, dept });
  };
```

```
return (
    <form onSubmit={handleSubmit}>
      <input type="text" value={name} onChange={(e) => setName(e.target.value)} />
      <input type="text" value={dept} onChange={(e) => setDept(e.target.value)} />
      <button type="submit">Add Employee</button>
    </form>
  );
}
```

2.3 COMPARISON TABLE

Feature	EMI Calculator (Frontend)	Employee Management System (Full-stack)
Tech Stack	HTML, CSS, JavaScript	React, Spring Boot, MySQL
Scope	Financial calculator	Employee & Department management
Architecture	Client-side only	Layered full-stack architecture
Data Storage	None (in-memory only)	Persistent storage in MySQL
Visualization	Pie chart, speedometer	Dynamic forms, dashboards planned
Challenges	JS logic, CSS styling	DTO alignment, backend integration

SQL (Structured Query Language)

SQL was a critical part of the **Employee Management System project**, as it handled all the data storage and retrieval operations.

Key Learnings:

- Creating and managing databases and tables for employees and departments.
- Using queries like SELECT, INSERT, UPDATE, and DELETE to perform CRUD operations.
- Applying constraints such as PRIMARY KEY and FOREIGN KEY to ensure data integrity.
- Performing joins to combine data from multiple tables.
- Using aggregate functions like COUNT, SUM, and AVG for analytics.

Example (Employee Table Schema):

```
CREATE TABLE employees (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100),
  department_id INT,
  FOREIGN KEY (department_id) REFERENCES departments(id)
);
```

Example Queries:

```
-- Insert new employee
INSERT INTO employees (name, department_id) VALUES ('Kathrin', 1);

-- Retrieve all employees
SELECT * FROM employees;

-- Update employee details
UPDATE employees SET name = 'Hepsiba' WHERE id = 2;

-- Delete employee
DELETE FROM employees WHERE id = 3;

-- Join employees with departments
SELECT e.name, d.dept_name
FROM employees e
JOIN departments d ON e.department_id = d.id;
```

SQL ensured that the backend system was reliable, scalable, and capable of handling multiple records efficiently.

React (Frontend Framework)

React was used to build the **Employee Management System frontend**. It allowed me to design dynamic, component-driven UIs.

Key Learnings:

- Building functional components with **JSX syntax**.
- Managing state using **useState** and handling side effects with **useEffect**.
- Implementing **React Router** for navigation between employee and department views.
- Creating controlled forms with real-time validation.
- Ensuring responsiveness and modularity with reusable components.

Example (Employee Form Component):

```
function EmployeeForm({ onSubmit }) {
  const [name, setName] = useState("");
  const [dept, setDept] = useState("");

  const handleSubmit = (e) => {
    e.preventDefault();
    onSubmit({ name, dept });
  };

  return (
    <form onSubmit={handleSubmit}>
      <input type="text" value={name} onChange={(e) => setName(e.target.value)} />
      <input type="text" value={dept} onChange={(e) => setDept(e.target.value)} />
      <button type="submit">Add Employee</button>
    </form>
  );
}
```

React ensured a smooth user experience with instant updates and no page reloads.

6. Spring Boot (Backend Framework)

Spring Boot was the backbone of the **Employee Management System backend**. It provided a layered architecture and RESTful APIs.

Key Learnings:

- Creating REST Controllers to handle HTTP requests.
- Implementing a **Service Layer** for business logic.
- Using **Spring Data JPA** for database interactions.

- Designing DTOs and Mappers for clean data transfer.
- Handling validation and error responses consistently.

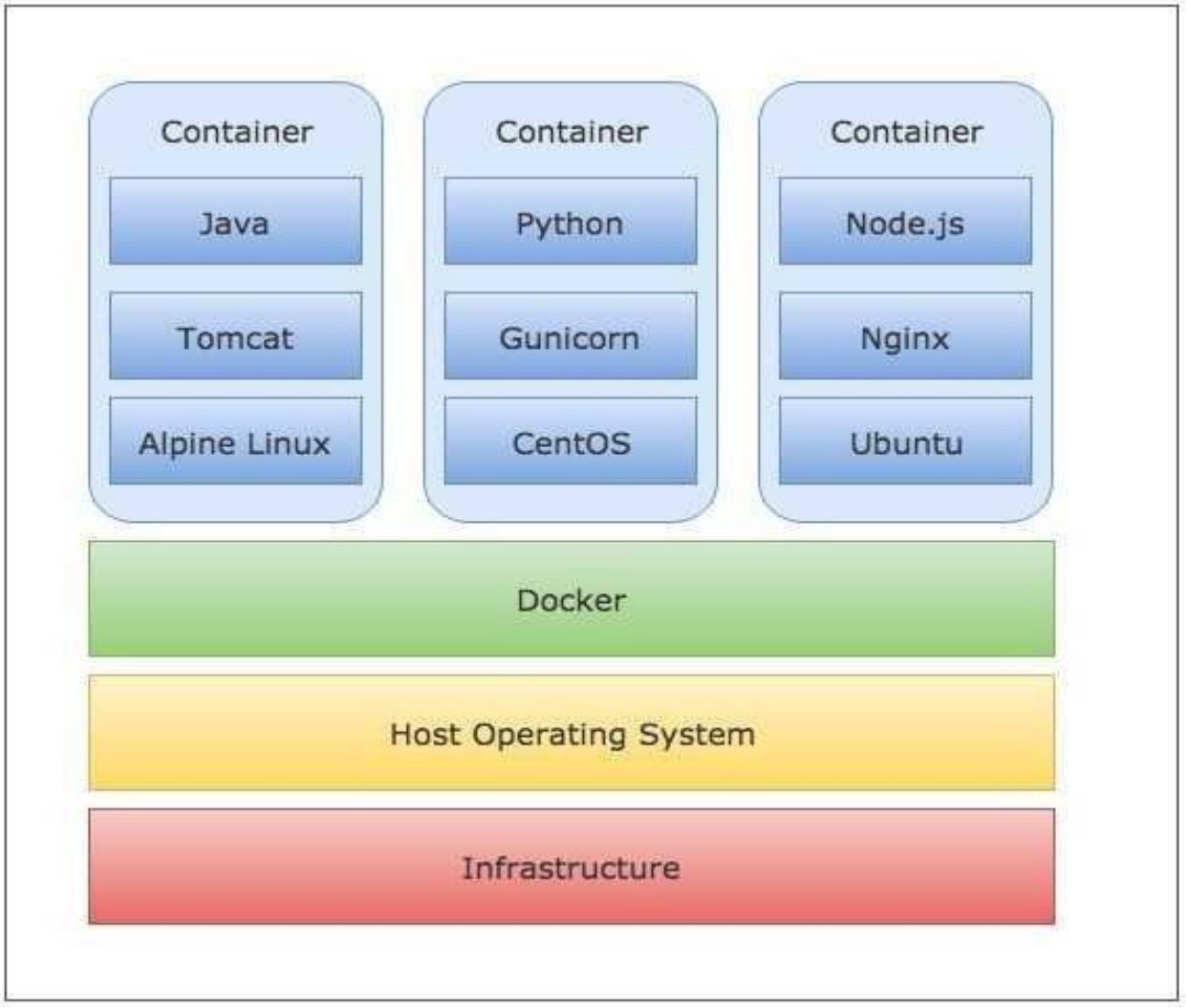


Figure 2.2 Spring Boot Project Development Life Cycle

- **Containers**: Each container encapsulates an application and its dependencies.
  - Example 1: Java + Tomcat on Alpine Linux
  - Example 2: Python + Gunicorn on CentOS
  - Example 3: Node.js + Nginx on Ubuntu
- **Docker Layer**: Provides the container runtime, ensuring isolation and portability.
- **Host Operating System**: The OS on which Docker runs (e.g., Linux, Windows).
- **Infrastructure**: The physical or virtual hardware resources (servers, cloud instances).

Example (REST Controller):

```
@RestController

@RequestMapping("/api/employees")

public class EmployeeController {

    @Autowired
    private EmployeeService service;

@GetMapping
    public List<Employee> getAllEmployees() {
        return service.getEmployees();
    }

    @PostMapping
    public Employee addEmployee(@RequestBody Employee emp) {
        return service.saveEmployee(emp);
    }
}
```

Spring Boot ensured scalability, modularity, and clean separation of concerns.

7. REST APIs

REST APIs were the communication bridge between React (frontend) and Spring Boot (backend).

Key Learnings:

- Designing endpoints using standard HTTP methods (GET, POST, PUT, DELETE).
- Sending JSON data between client and server.
- Handling request/response cycles with proper status codes.
- Ensuring predictable and intuitive API behavior.

Example Endpoints:

- GET /api/employees → Retrieve all employees.
- POST /api/employees → Add a new employee.
- PUT /api/employees/{id} → Update employee details.
- DELETE /api/employees/{id} → Delete employee by ID.

REST APIs ensured smooth integr

ation between frontend and backend.

2.4 RESTful Web Services with Spring Boot

REST (Representational State Transfer) is an architectural style used to design scalable and lightweight web services. In your internship projects, REST APIs acted as the **bridge between Angular frontend and Spring Boot backend**, enabling smooth communication.

1. Key Concepts of REST

- **Resources**: Data entities (e.g., posts, users, products) exposed via endpoints.
- **HTTP Methods**:
  - GET → Retrieve data
  - POST → Create new data
  - PUT → Update existing data
  - DELETE → Remove data
- **Statelessness**: Each request is independent; the server does not store client state.
- **JSON/XML Responses**: Lightweight formats for data exchange.

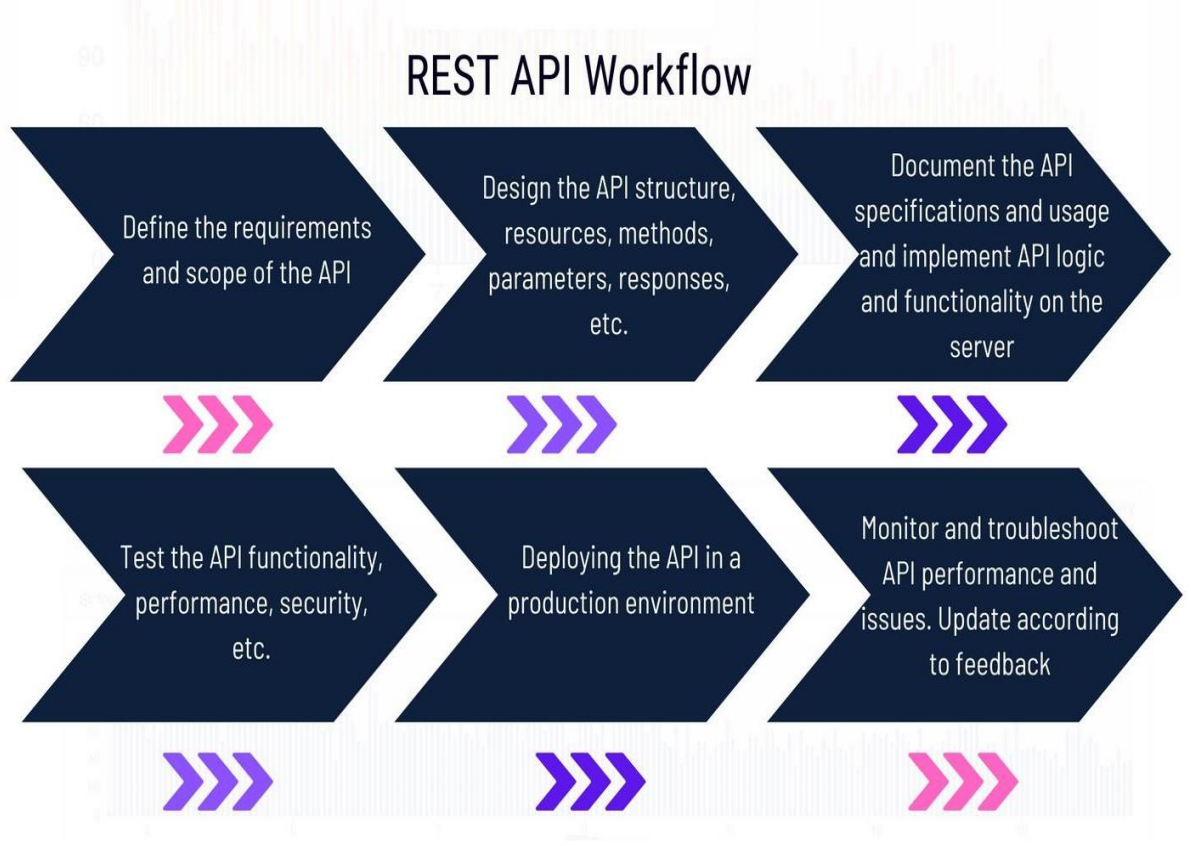


Figure 2.3 REST API Development Workflow

2. Example – REST Controller in Spring Boot

```
@RestController
@RequestMapping("/api/community")
public class CommunityController {

    @Autowired
    private CommunityService service;

    // Create a new post
    @PostMapping
    public CommunityPostDTO createPost(@RequestBody CommunityPostDTO dto) {
        return service.savePost(dto);
    }

    // Retrieve all posts
    @GetMapping
    public List<CommunityPostDTO> getAllPosts() {
        return service.getAllPosts();
    }
}
```

3. Angular Service for API Calls

```
@Injectable({ providedIn: 'root' })
export class CommunityService {
  private apiUrl = 'http://localhost:8080/api/community';

  constructor(private http: HttpClient) {}

  // Fetch posts
  getPosts(): Observable<CommunityPost[]> {
    return this.http.get<CommunityPost[]>(this.apiUrl);
  }

  // Add new post
  addPost(post: CommunityPost): Observable<CommunityPost> {
```

```
    return this.http.post<CommunityPost>(this.apiUrl, post);
  }
}
```

4. REST API Lifecycle

- **Request**: Angular sends HTTP request to backend.
- **Processing**: Spring Boot controller handles request, interacts with service and database.
- **Response**: Backend returns JSON data to frontend.
- **Rendering**: Angular updates UI dynamically with the received data.

5. Advantages of REST in Full Stack Development

- **Scalability**: APIs can handle multiple clients (web, mobile).
- **Flexibility**: Frontend and backend can evolve independently.
- **Portability**: Works across different platforms and languages.
- **Integration**: Easy to connect with third-party services.

6. Real-Time Examples from Internship

- **Community Feature**: REST APIs handled post submission and retrieval.
- **Eat Fit Blog Website**: Blog entries stored in backend and displayed via Angular.
- **Bank Management System**: Account creation, balance check, and transaction history exposed as REST endpoints.

Angular

Angular is a powerful, TypeScript-based development platform and framework for building scalable, single-page web applications (SPAs). It is maintained by Google and is widely used for creating robust, enterprise-level frontends.

Why Angular?

- **Component-Based Architecture:** Encourages modularity and code reusability.
- **Two-Way Data Binding:** Synchronizes data between the model and the view automatically.
- **TypeScript Support:** Provides static typing and better tooling, reducing runtime errors.
- **Built-in Features:** Includes powerful tools for routing, form handling, and HTTP client services out of the box.
- **Scalability:** Highly suitable for large-scale, complex applications.

Key Features

- **Angular CLI:** A command-line interface that simplifies project creation, testing, and deployment.
- **Directives:** Extend HTML functionality with custom behavior (e.g., *ngIf, *ngFor).
- **Dependency Injection:** Efficiently manages object creation and provides services to components.
- **RxJS Integration:** Uses Reactive Extensions for powerful asynchronous data stream management.

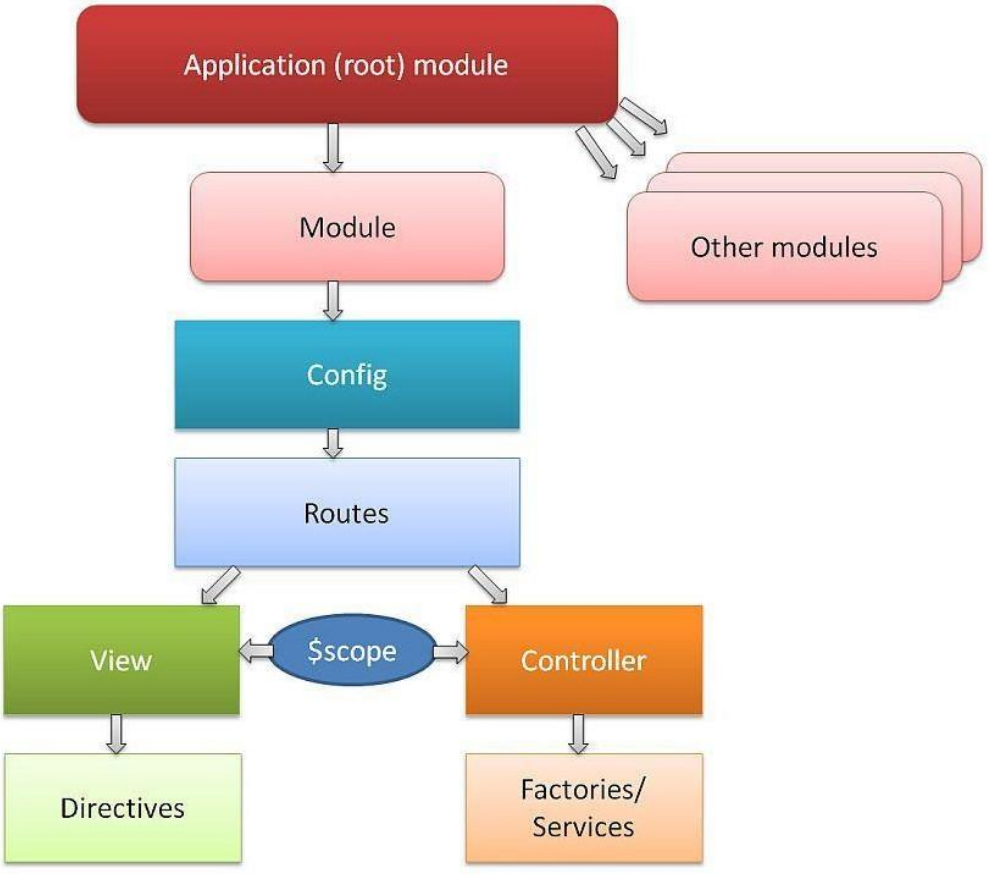


Figure 2.4 Angular Application Architecture and Component Hierarchy

Important Topics in Angular

1. Components

The fundamental building block of an Angular UI. Each component consists of an HTML template, a CSS stylesheet, and a TypeScript class for logic.

Example:

TypeScript

```
@Component({
selector: 'app-root',
  templateUrl: './app.component.html'
})
export class AppComponent { title = 'EmployeePortal'; }
```

2. Modules (NgModule)

Used to group related components, directives, and services into functional blocks. Every app has at least one root module (AppModule).

3. Data Binding

Defines how the view and the component communicate.

- **Interpolation {{ }}:** Displays logic data in the UI.
- **Property Binding [ ]:** Sets element properties.
- **Event Binding ( ):** Listens for user actions like clicks.
- **Two-way Binding [(ngModel)]:** Combines property and event binding for form inputs.

4. Services & Dependency Injection (DI)

Services handle business logic and data fetching, keeping components lean. DI allows these services to be shared across the application.

Example:

TypeScript

```
@Injectable({ providedIn: 'root' })
export class EmployeeService { ... }
```

5. Directives

- **Structural Directives:** Modify the DOM layout (e.g., *ngIf for conditions, *ngFor for lists).
- **Attribute Directives:** Change the appearance or behavior of an element (e.g., ngClass).

6. Routing and Navigation

The Angular Router enables navigation between different views or pages without a full browser refresh, essential for Single Page Applications (SPAs).

7. Forms (Reactive & Template-driven)

Angular provides two ways to handle user input. Reactive forms are preferred for complex logic as they are more robust and scalable.

8. HTTP Client

A built-in service (HttpClient) used to communicate with backend REST APIs (like Spring Boot) to fetch or send data.

9. Pipes

Used to transform data directly in the template (e.g., formatting dates, currency, or filtering lists).

10. Lifecycle Hooks

Angular provides hooks like ngOnInit and ngOnDestroy to tap into key moments of a component's life, from creation to destruction.

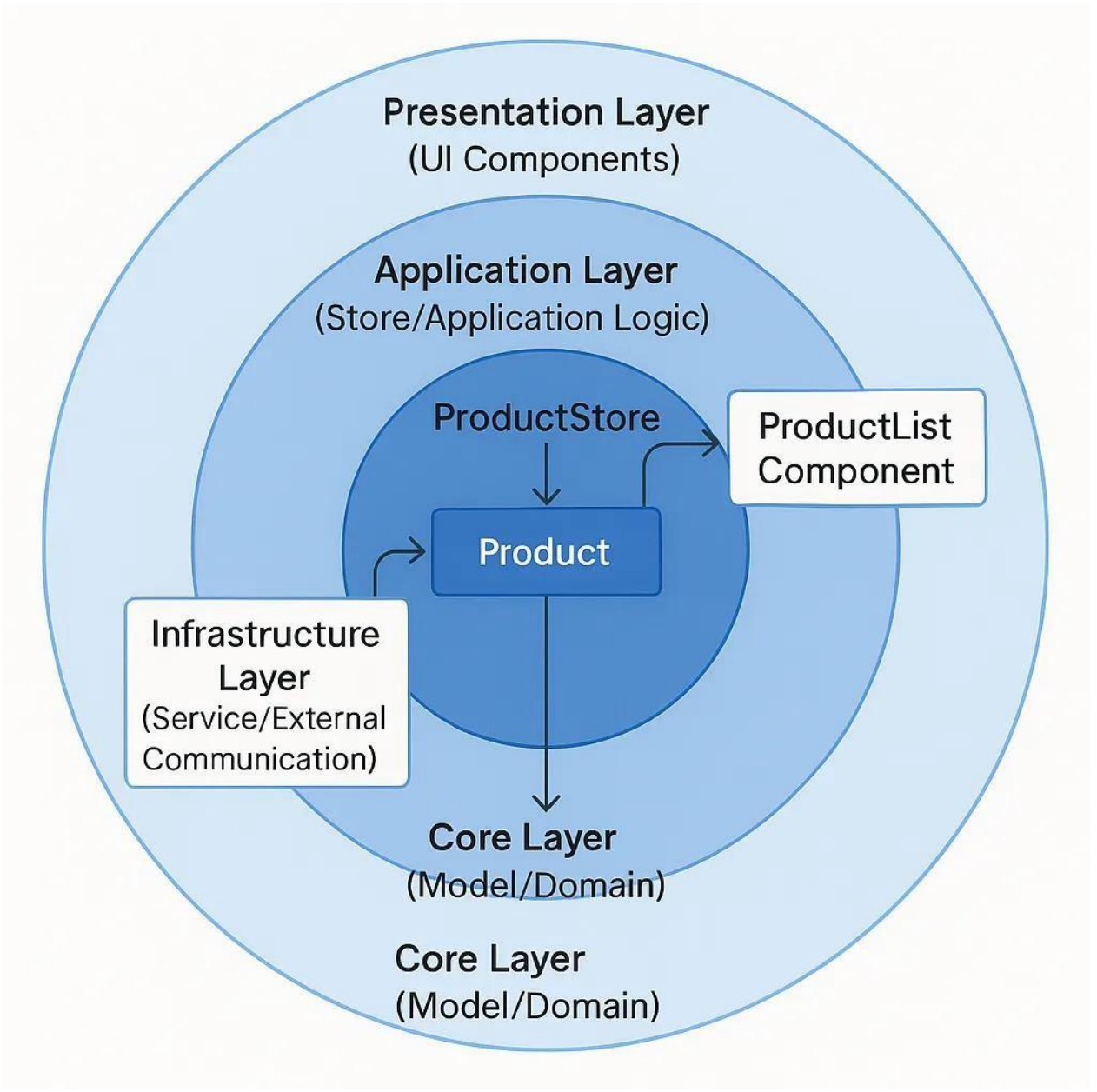


Figure 2.5 Layered Onion Architecture of the Web Application

Conclusion

Angular simplifies frontend development by providing a structured framework and a suite of integrated tools, making it ideal for building dynamic, high-performance web applications.

In summary, Full Stack Development using Angular and Spring Boot integrates modern UI/UX principles with robust backend logic. By leveraging Angular's modular architecture and Spring Boot's efficient API management, developers can build seamless, production-ready systems like the **Employee Management System** and **EMI Calculator** that meet modern business standards.

Five Phases of Full Stack Development (Angular Focus):

1. Concept and Design:

- **UI/UX Wireframing:** Designing the modular layout of Angular components using tools like Figma to ensure a responsive user experience.
- **Component Mapping:** Defining which parts of the application will be reusable components (e.g., Navbar, Sidebars, Input Cards).

2. Development:

- **Frontend Development:** Building the UI using **Angular**, **HTML5**, and **SCSS/CSS**. Implementing routing and state management to ensure a smooth flow between pages.
- **API Integration:** Connecting the Angular frontend to the **Spring Boot** backend using the HttpClient module to perform CRUD operations on employee and loan data.

3. Prototype Development:

- Creating working versions of the EMI Calculator and Employee lists. Using **Postman** to verify that the Angular service correctly consumes the JSON data from the backend.

4. Refinement Based on Feedback:

- Optimizing component loading times and refining form validations (e.g., ensuring interest rates in the EMI calculator are within valid ranges) based on user testing.

5. Deployment and Maintenance:

- **Packaging:** Building the production-ready Angular app using ng build --prod.
- **Hosting:** Deploying the application on cloud platforms or local servers, ensuring that the frontend and backend are correctly synchronized in the production environment

SERVICE AND SUPPORT

5.1 Technical Support and Monitoring

In the lifecycle of web applications like the **Employee Management System** and the **EMI Calculator**, the development phase is followed by a rigorous support and monitoring phase. This ensures that the applications remain stable, performant, and responsive to user needs.

Logging and Monitoring Frameworks:

During my internship, I learned that tracking application behavior is critical for maintaining high availability.

- **ELK Stack (Elasticsearch, Logstash, Kibana):** * **Elasticsearch:** Acts as the search and analytics engine for storing application logs.
  - **Logstash:** Processes and transforms log data before sending it to Elasticsearch.
  - **Kibana:** Provides a visualization dashboard to track user traffic, server errors (404/500), and API performance for the Spring Boot backend.
- **Prometheus and Grafana:** These tools are used for real-time monitoring of system metrics. In my projects, this involves monitoring memory usage of the Java Virtual Machine (JVM) and tracking the response times of the REST API endpoints to prevent system lag.

5.2 Updates and Bug Fixes

Web applications require continuous refinement to address security vulnerabilities and improve user experience. My project work followed an iterative process for updates:

- **Performance Improvements:** For the **EMI Calculator**, updates focused on optimizing JavaScript mathematical logic to ensure instant results even on lower-end devices. For the **Employee Management System**, database indexing was refined to speed up record retrieval.
- **Security Patches:** This involves regular updates to **Spring Boot** dependencies and **Angular/React** libraries to protect against common web vulnerabilities like Cross-Site Scripting (XSS) or SQL Injection.
- **Agile and Scrum Methodologies:** I utilized these methodologies to manage incremental releases. This involved breaking down new features into user stories and executing them in 2-week sprints, ensuring that bug fixes were prioritized based on user feedback.

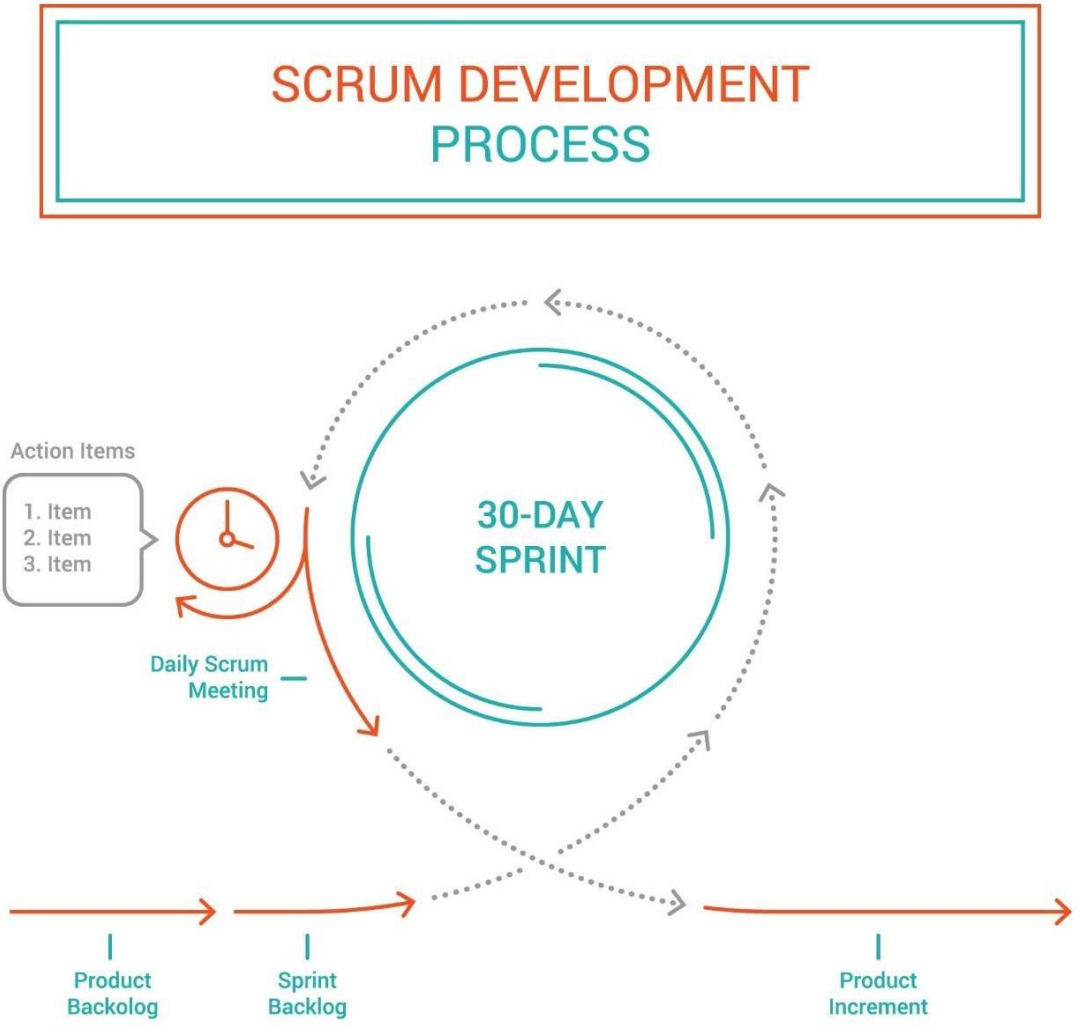


Figure 2.6 Agile Scrum Framework for Project Management

CONCLUSION

The completion of the **EMI Calculator** and **Employee Management System** marks a significant transition from theoretical knowledge to practical web development expertise.

1. **Technical Proficiency:** I have successfully integrated frontend frameworks (**Angular/React**) with a robust **Java Spring Boot** backend and a **MySQL** database, mastering the full 3-tier architecture.
2. **Problem Solving:** Through the development process, I learned to handle complex challenges such as state management in the frontend, RESTful API synchronization, and relational data integrity.

3. **Industry Readiness:** By following the formal **Phases of Development** and adopting **Service and Support** best practices, I have gained the skills necessary to build, deploy, and maintain professional-grade web applications.

In summary, the integration of modern tools like **Spring Data JPA**, **REST APIs**, and **Component-based UI design** has resulted in scalable and efficient solutions that align with current industry standards in Full Stack Web Development.

Benefits of Web Development Support Lifecycle

- **Stability:** Continuous monitoring ensures that the Employee Management System is always accessible to HR admins.
- **Reliability:** Regular bug fixes in the EMI Calculator logic maintain the accuracy of financial results.
- **Security:** Patching the tech stack prevents unauthorized access to sensitive employee data.
- **Scalability:** Agile updates allow the application to grow with new features without compromising the existing codebase.

CHAPTER 3: PROJECTS/MODULES COMPLETED

3. MODULE/PROJECT COMPLETED

3.1 Web Applications - EMI Calculator and Employee Management System

Objective:

The objective of this project is to develop two web-based applications: an EMI Calculator and an Employee Management System. These applications aim to simulate real-world digital services through interactive user interfaces, providing hands-on experience in full-stack development, API integration, and dynamic data visualization. The projects are built using **Full Stack Java (Spring Boot)** and modern frontend technologies like **React/Angular**, focusing on implementing advanced concepts like RESTful APIs, Component-based architecture, and Relational Database Management.

Description:

1. EMI Calculator Web Application

The EMI Calculator is a financial tool designed to provide users with instant loan repayment insights where users can:

- **Adjust Loan Parameters:** Use interactive sliders to set loan amount, interest rate, and tenure.
- **Visualize Financial Data:** View a breakdown of principal vs. interest through graphical charts.
- **Assess Loan Risk:** Check a risk "speedometer" to determine the feasibility of the loan.
- **Manage Calculation History:** Save results and view a history snapshot of previous sessions.

The main features of the application include:

- **Interactive Dashboard:**
  - The system displays input sliders and real-time output fields for monthly EMI, total interest, and total payment.
- **Dynamic Visualization:**
  - Integrates a **Pie Chart** that updates instantly as sliders move, showing the ratio of interest to principal.
- **Validation & Logic:**
  - Uses JavaScript to implement complex financial formulas and ensures no invalid inputs (like zero interest) are processed.
- **Persistence Feature:**
  - A "Save" functionality allows users to store their specific calculations for future reference.

2. Employee Management System (EMS)

The Employee Management System is a full-stack CRUD application designed to manage organizational records. It allows users to:

- **View Records:** List all employees and their respective departments.
- **Manage Data:** Add new employees, update existing profiles, or remove records.
- **Search & Filter:** Find specific employees or organize data by department.
- **Sync Data:** Ensure the frontend UI and backend database are always synchronized via REST APIs.

The main features of the application include:

- **Full CRUD Functionality:**
  - The system provides forms for creating and editing employee details (Name, Email, Role) and department categories.
- **Layered Architecture:**
  - Built using a **Controller-Service-Repository** pattern in Spring Boot to ensure clean code and scalability.
- **Database Integration:**
  - Utilizes **MySQL** and **Spring Data JPA** to store data permanently. The application handles entity relationships (One-to-Many) between departments and employees.
- **Frontend-Backend Communication:**
  - Uses an HTTP client to consume RESTful endpoints, ensuring that data updates in the browser happen seamlessly without full page reloads.

Tech Stack for Both Projects:

Technology	Purpose
Java (Core)	Programming language used for developing logic and control flow
Arrays/ArrayLists	Data structures used for storing products and cart items
Methods	Functions used to implement actions like adding/removing items, selecting products, etc.
Control Flow	Loops, conditionals used to manage user inputs and application flow
Scanner	User input management for product selection, payment, etc.

3.2 Concepts Demonstrated

Object-Relational Mapping (ORM) with Hibernate:

The project utilizes **Spring Data JPA** (powered by Hibernate) to bridge the gap between Java objects and the **MySQL** relational database. This allows the application to perform complex database operations without writing manual SQL queries, ensuring that the **Employee Management System** is maintainable and the code remains "database agnostic."

Database Normalization and Integrity:

The **MySQL** schema is designed following **Normalization** principles (up to 3rd Normal Form) to minimize data redundancy. By using **Primary Keys** and **Foreign Keys**, the system ensures **Referential Integrity**, meaning every employee record is correctly mapped to an existing department, preventing data corruption.

RESTful Web Services and HTTP Verbs:

The backend logic is exposed through a **REST (Representational State Transfer)** architecture. It utilizes standard HTTP methods—**GET** to retrieve employee lists, **POST** to add new hires, **PUT** to update details, and **DELETE** to remove records. This ensures a standardized communication protocol between the React/Angular frontend and the Spring Boot server.

Dependency Injection (DI) and Inversion of Control (IoC):

By leveraging the **Spring IoC Container**, the application achieves "loose coupling." Using the @Autowired annotation, the system automatically manages the lifecycle of objects and injects required dependencies into controllers. This makes the code modular, easier to test, and highly scalable.

Single Page Application (SPA) Routing:

To enhance user experience, the frontend utilizes **Client-Side Routing** (React Router or Angular Router). This allows users to navigate between the **EMI Calculator** and the **EMS Dashboard** instantly without a full page reload, providing a seamless, "app-like" performance in the web browser.

Asynchronous Data Fetching:

The application demonstrates **Asynchronous Programming** using the Fetch API or Axios. This ensures that while the frontend is waiting for a response from the backend server, the UI remains interactive and doesn't freeze, which is a critical requirement for modern web-based systems.

Cross-Origin Resource Sharing (CORS) Management:

Since the frontend and backend operate on different local servers during development, **CORS policies** were configured in Spring Boot. This concept demonstrates a deep understanding of web security, allowing the browser to safely exchange data across different origins.

Layered Service Pattern:

The business logic is encapsulated within a dedicated **Service Layer**, separate from the **Controller** and **Repository**. This separation of concerns ensures that the logic for calculating interest in the **EMI tool** is centralized and can be reused or modified without affecting the API endpoints or database access code.

Input Validation and Data Sanitization:

A two-tier validation strategy is implemented. **Client-side validation** provides immediate feedback to the user (e.g., preventing empty names), while **Server-side validation** (using JSR-303/Bean Validation) ensures that only clean, sanitized data enters the database, protecting the system from malicious inputs.

Responsive Web Design (RWD):

The **EMI** and the **EMS** utilize **CSS Media Queries** and **Grid/Flexbox** systems. This ensures that the application layout adjusts dynamically to various screen sizes, providing a consistent user experience across desktops, tablets, and mobile devices.

State Management and Data Binding:

In the **EMI Calculator**, the project demonstrates **Two-Way Data Binding**. As the user moves the sliders for loan amounts, the application's "state" is updated instantly, which in turn triggers a re-render of the **Pie Chart** and **Speedometer** without requiring any manual page updates.

3.3 About Project 1

Project Title: EMI Calculator – A Responsive Web Application

Objective

The objective of this project is to design and implement a **responsive EMI Calculator** that allows users to compute monthly loan installments based on principal amount, interest rate, and tenure. The project demonstrates **front-end web development skills** using **HTML5, CSS3, and JavaScript**, focusing on:

- User input validation
- Dynamic calculations using mathematical formulas
- DOM manipulation for real-time output
- Responsive design for cross-device usability

Problem Statement

Traditional EMI calculations are often done manually or using offline spreadsheets, which can be error-prone and inconvenient. A modern web-based EMI Calculator provides:

- **Instant results** without manual effort
- **User-friendly interface** accessible on any device
- **Accurate financial planning** for loans and investments

System Architecture

The EMI Calculator follows a **three-layer architecture**:

1. **Presentation Layer (Frontend)**
    - Built using **HTML5 and CSS3**.
    - Responsible for input forms, buttons, and result display.
    - Provides responsive design using Flexbox and media queries.
2. **Application Layer (Logic)**
    - Implemented in **JavaScript**.
    - Handles EMI formula calculation.
    - Validates user inputs and updates DOM dynamically.
3. **Data Layer**
    - No external database used.
    - User inputs are processed in-memory.
    - Results are displayed instantly without storage.

Description of Features

1. **Input Form**
   - Fields for Loan Amount, Interest Rate, and Tenure.
   - Validation ensures numeric input only.
2. **Calculation Logic**
   - EMI formula implemented in JavaScript:

     [EMI = \frac{P \cdot R \cdot (1+R)^N}{(1+R)^N - 1}]

     where:
   - (P) = Principal
   - (R) = Monthly Interest Rate
   - (N) = Tenure in months
3. **Result Display**
   - Shows Monthly EMI and Total Payable Amount.
   - Updates dynamically when user clicks “Calculate”.
4. **Responsive Design**
   - Layout adapts to desktop, tablet, and mobile screens.
   - CSS media queries ensure usability across devices.

Sample Code Snippets

HTML Form:

```
<form>
  <label>Loan Amount:</label>
  <input type="number" id="amount" required>

  <label>Interest Rate (% per annum):</label>
  <input type="number" id="rate" required>

  <label>Tenure (months):</label>
  <input type="number" id="tenure" required>

  <button type="button" onclick="calculateEMI()">Calculate</button>
</form>
<div id="result"></div>
```

JavaScript Logic:

```
function calculateEMI() {
  let principal = parseFloat(document.getElementById("amount").value);
  let rate = parseFloat(document.getElementById("rate").value) / 100 / 12;
  let tenure = parseInt(document.getElementById("tenure").value);

  let emi = (principal * rate * Math.pow(1 + rate, tenure)) /
            (Math.pow(1 + rate, tenure) - 1);

  let total = emi * tenure;

  document.getElementById("result").innerHTML  =
    `Monthly EMI: ₹${emi.toFixed(2)} <br> Total Payable: ₹${total.toFixed(2)}`;
}
```

CSS Styling:

```
form {
  max-width: 400px;
  margin: auto;
  padding: 20px;
  background: #f4f4f4;
  border-radius: 8px;
}
button {
  background: #007bff;
  color: white;
  padding: 10px;
  border: none;
  cursor: pointer;
}
button:hover {
  background: #0056b3;
}
```

Challenges Faced

- Handling floating-point precision in EMI calculations.
- Designing a responsive layout that works across devices.
- Ensuring proper validation for empty or invalid inputs.

Learning Outcomes

- Mastery of **JavaScript DOM manipulation**.
- Understanding of **financial formulas** in real-world applications.
- Improved skills in **responsive web design** using CSS media queries.
- Experience in building **interactive front-end applications** without backend dependency.

OUTPUT

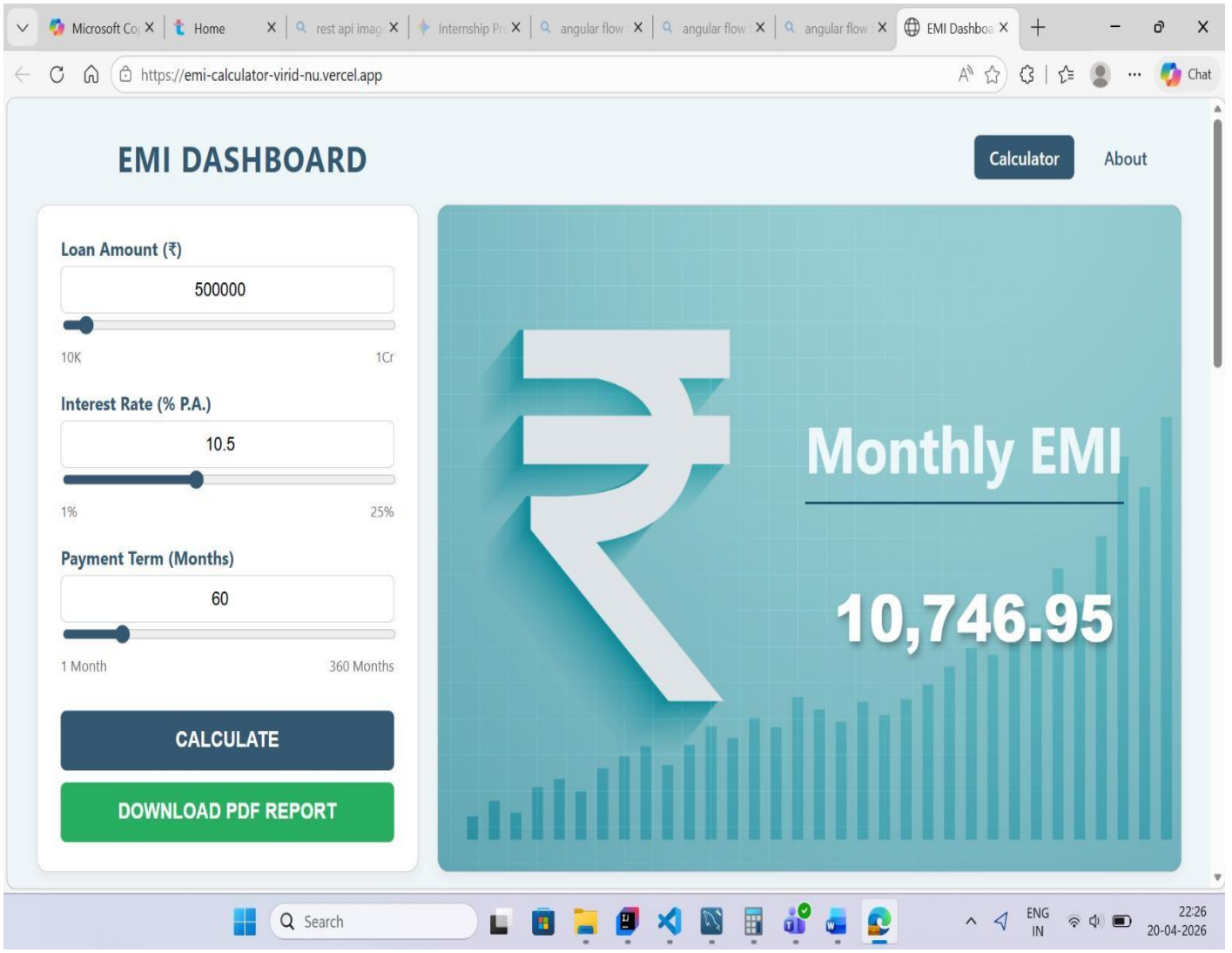


Figure 3.1 Input from user and calculating the monthly EMI

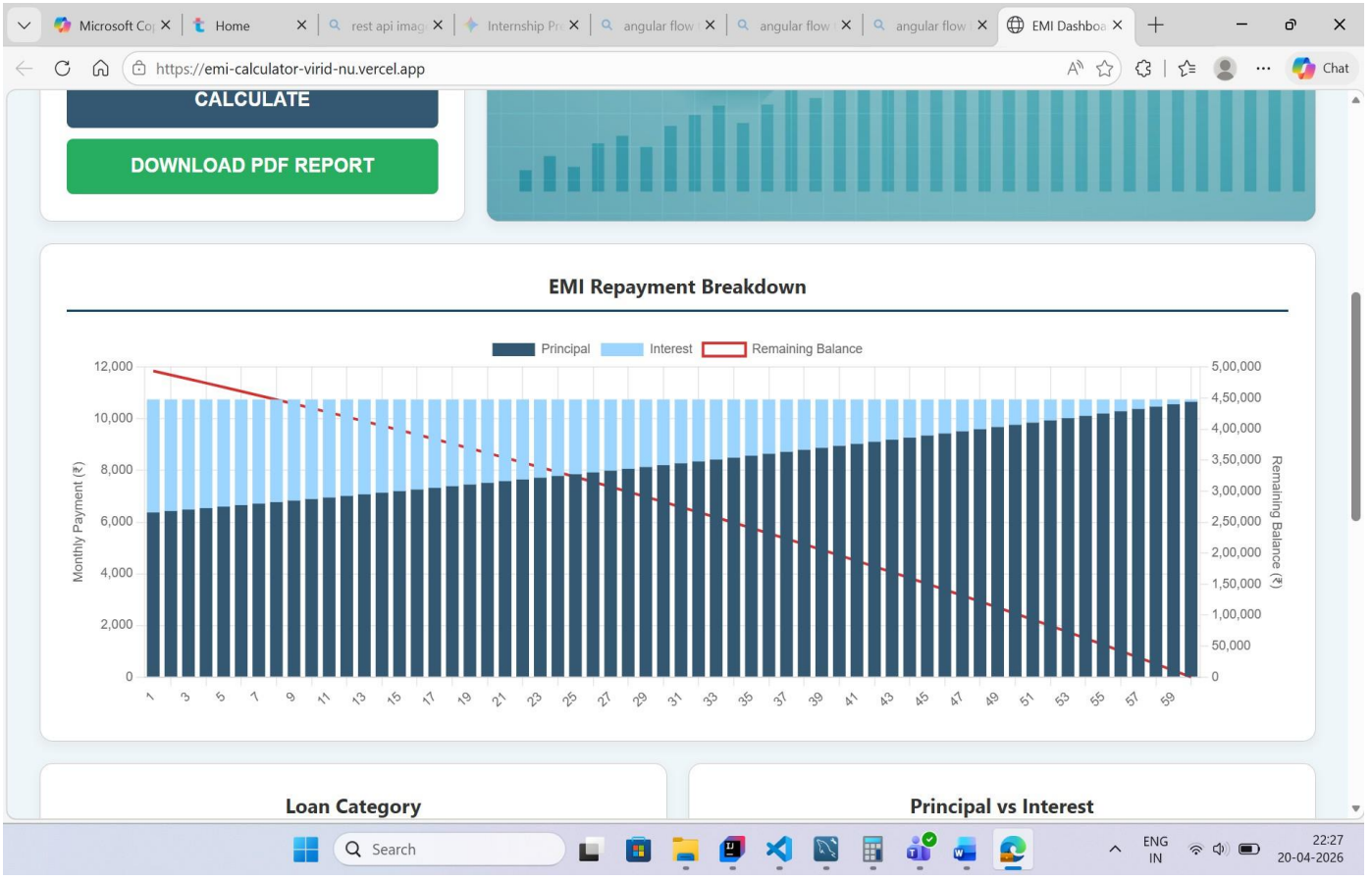


Figure 3.2 Bar Graph Representation of the emi over each month

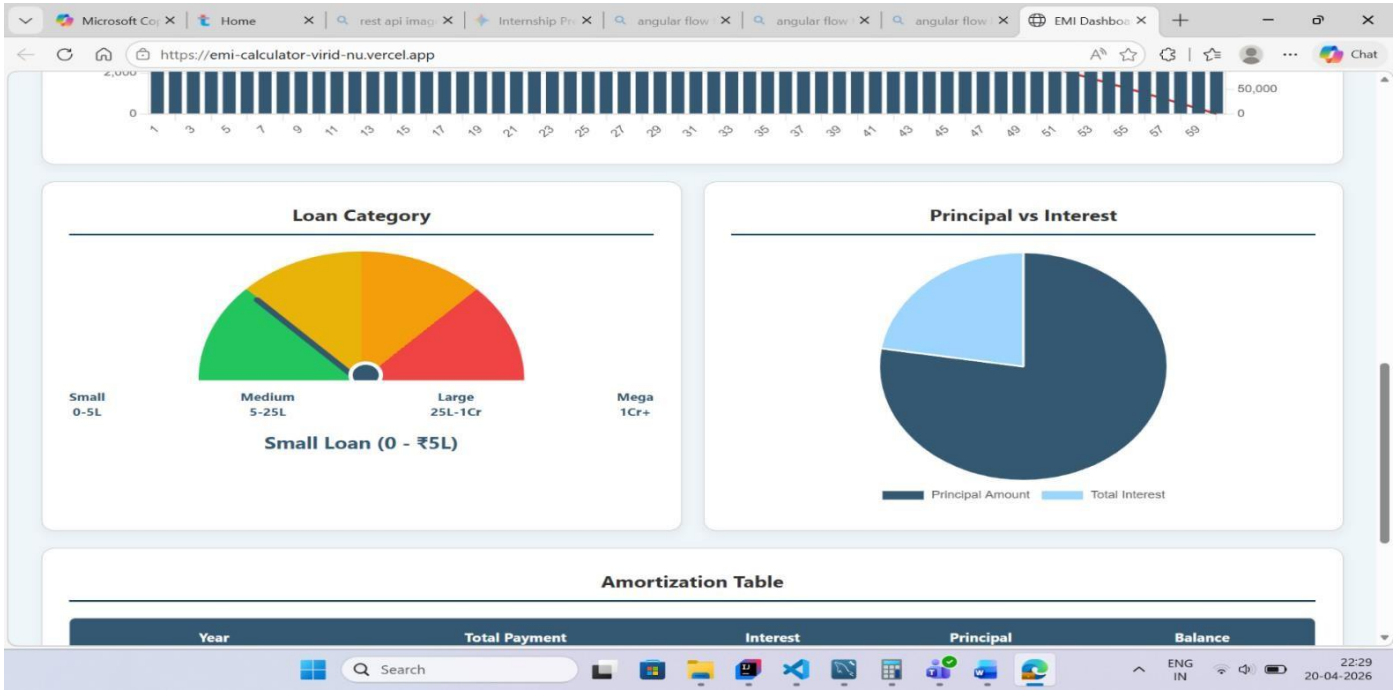
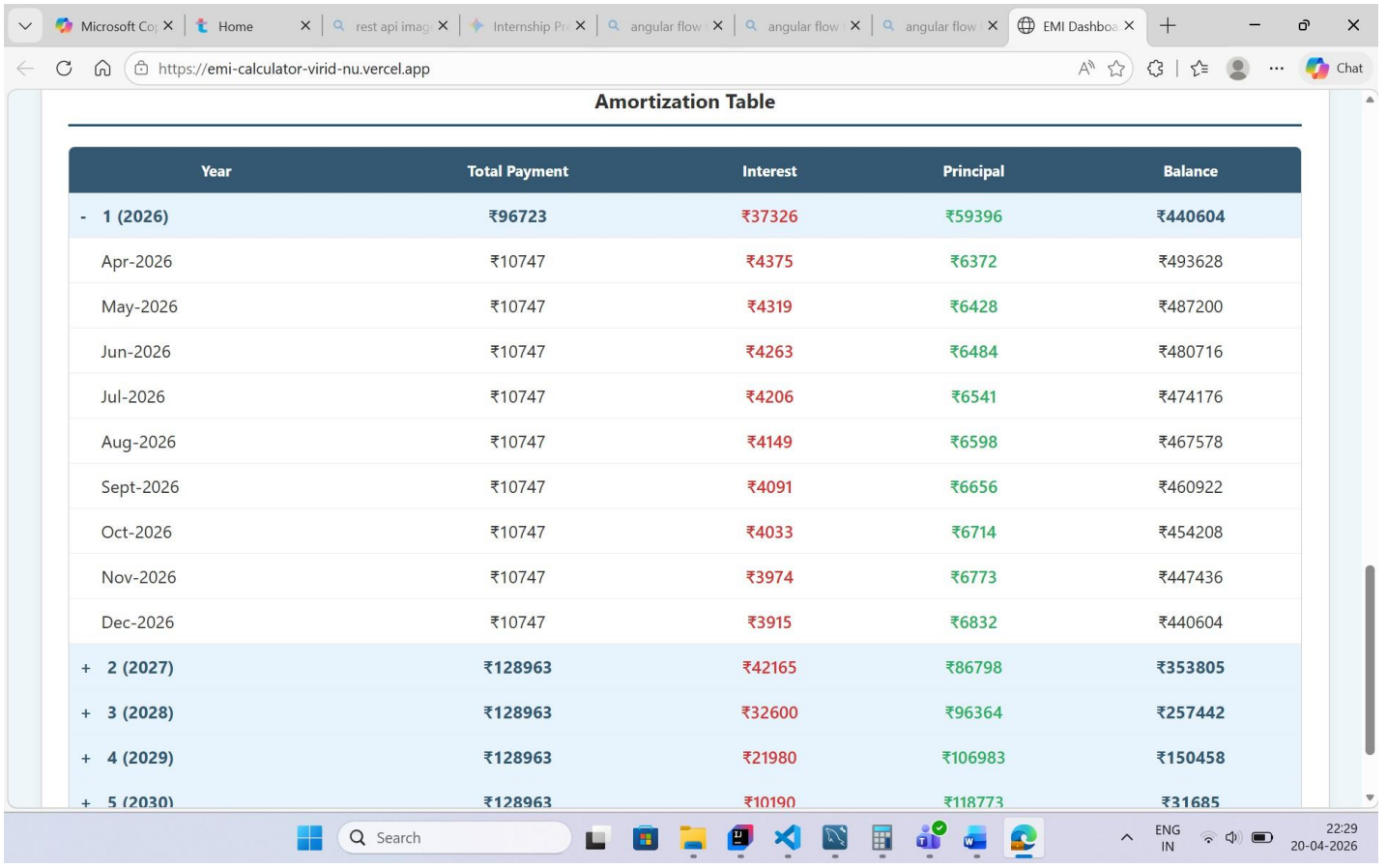


Figure 3.3 Pictorial representation of monthly EMI



Year	Total Payment	Interest	Principal	Balance
- 1 (2026)	₹96723	₹37326	₹59396	₹440604
Apr-2026	₹10747	₹4375	₹6372	₹493628
May-2026	₹10747	₹4319	₹6428	₹487200
Jun-2026	₹10747	₹4263	₹6484	₹480716
Jul-2026	₹10747	₹4206	₹6541	₹474176
Aug-2026	₹10747	₹4149	₹6598	₹467578
Sept-2026	₹10747	₹4091	₹6656	₹460922
Oct-2026	₹10747	₹4033	₹6714	₹454208
Nov-2026	₹10747	₹3974	₹6773	₹447436
Dec-2026	₹10747	₹3915	₹6832	₹440604
+ 2 (2027)	₹128963	₹42165	₹86798	₹353805
+ 3 (2028)	₹128963	₹32600	₹96364	₹257442
+ 4 (2029)	₹128963	₹21980	₹106983	₹150458
+ 5 (2030)	₹128963	₹10190	₹118773	₹31685

Figure 3.4 Amortization table of monthly emi

3.4 About Project 2

Project Title: Employee Management System – Full Stack Web Application

Objective

The objective of this project is to design and implement a **full-stack Employee Management System (EMS)** that enables organizations to manage employee records efficiently. The system supports CRUD (Create, Read, Update, Delete) operations, integrates a responsive frontend with a robust backend, and ensures secure data storage in a relational database.

Problem Statement

Traditional employee record management often relies on manual processes or outdated systems, leading to inefficiency, duplication, and errors. A modern web-based EMS provides:

- **Centralized employee data management**
- **Real-time updates** across frontend and backend
- **Scalable architecture** for future enhancements
- **User-friendly interface** for HR and admin staff

System Architecture

The EMS follows a **three-tier architecture**:

1. **Presentation Layer (Frontend – Angular)**
    - Provides forms for employee data entry.
    - Displays employee lists with search and filter options.
    - Responsive UI using Angular Material and CSS.
2. **Application Layer (Backend – Spring Boot)**
    - REST APIs handle employee CRUD operations.
    - Service layer implements business logic.
    - Exception handling ensures smooth execution.
3. **Data Layer (Database – MySQL)**
    - Employee table stores records with fields like id, name, designation, salary.
    - Spring Data JPA manages ORM mapping.
    - Queries executed via repositories.

Description of Features

1. **Employee Registration**
    - Add new employees with details like name, designation, and salary.
    - Input validation ensures correct data entry.

2. **Employee Listing**
    - Displays all employees in a tabular format.
    - Supports search and filter functionality.
3. **Update Employee Records**
    - Allows editing of existing employee details.
    - Updates reflected instantly in the database.
4. **Delete Employee Records**
    - Provides option to remove employees from the system.
    - Ensures data consistency with proper checks.

Sample Code Snippets

Angular Service (Frontend):

```
@Injectable({ providedIn: 'root' })
export class EmployeeService {
  private apiUrl = 'http://localhost:8080/api/employees';

  constructor(private http: HttpClient) {}

  getEmployees(): Observable<Employee[]> {
    return this.http.get<Employee[]>(this.apiUrl);
  }

  addEmployee(emp: Employee): Observable<Employee> {
    return this.http.post<Employee>(this.apiUrl, emp);
  }

  updateEmployee(emp: Employee): Observable<Employee> {
    return this.http.put<Employee>(`${this.apiUrl}/${emp.id}`, emp);
  }

  deleteEmployee(id: number): Observable<void> {
    return this.http.delete<void>(`${this.apiUrl}/${id}`);
  }
}
```

Spring Boot REST Controller (Backend):

```
@RestController
@RequestMapping("/api/employees")
public class EmployeeController {

    @Autowired
    private EmployeeService service;

    @GetMapping
    public List<Employee> getAllEmployees() {
        return service.getAllEmployees();
    }

    @PostMapping
    public Employee addEmployee(@RequestBody Employee emp) {
        return service.saveEmployee(emp);
    }

    @PutMapping("/{id}")
    public Employee updateEmployee(@PathVariable int id, @RequestBody Employee emp) {
        return service.updateEmployee(id, emp);
    }

    @DeleteMapping("/{id}")
    public void deleteEmployee(@PathVariable int id) {
        service.deleteEmployee(id);
    }
}
```

Database Schema (MySQL):

```
CREATE TABLE employee (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,
  designation VARCHAR(50),
  salary DECIMAL(10,2)
```

);

Challenges Faced

- Integrating Angular frontend with Spring Boot backend.
- Handling CORS issues during API calls.
- Designing a responsive UI for multiple screen sizes.
- Ensuring proper database connectivity and exception handling.

Learning Outcomes

- Mastery of **Angular services and HTTP client** for API communication.
- Understanding of **RESTful API design** with Spring Boot.
- Practical experience with **Spring Data JPA and MySQL integration**.
- Improved skills in **full-stack application development**.
- Ability to implement **real-world CRUD operations** with proper validation.

OUTPUT

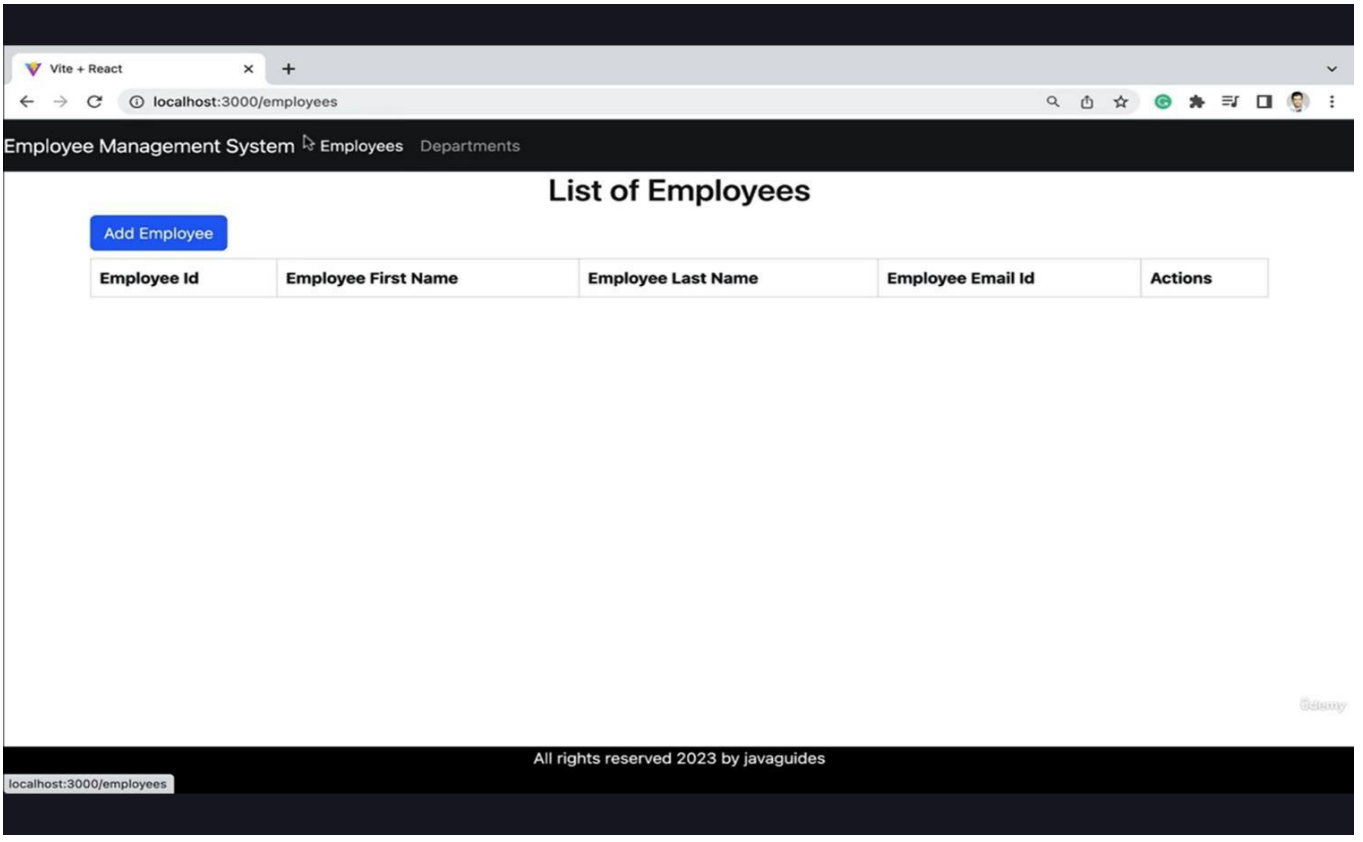


Figure 3.5 Listing of employees

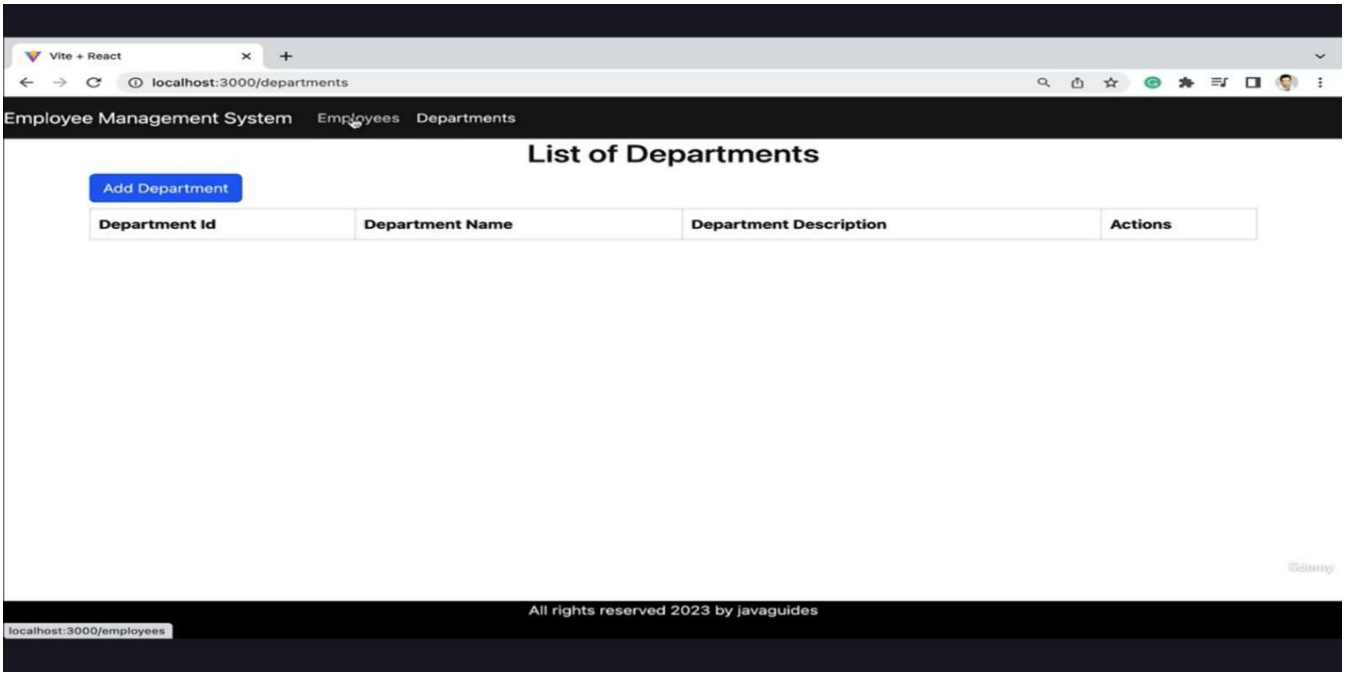


Figure 3.6 Listing of department

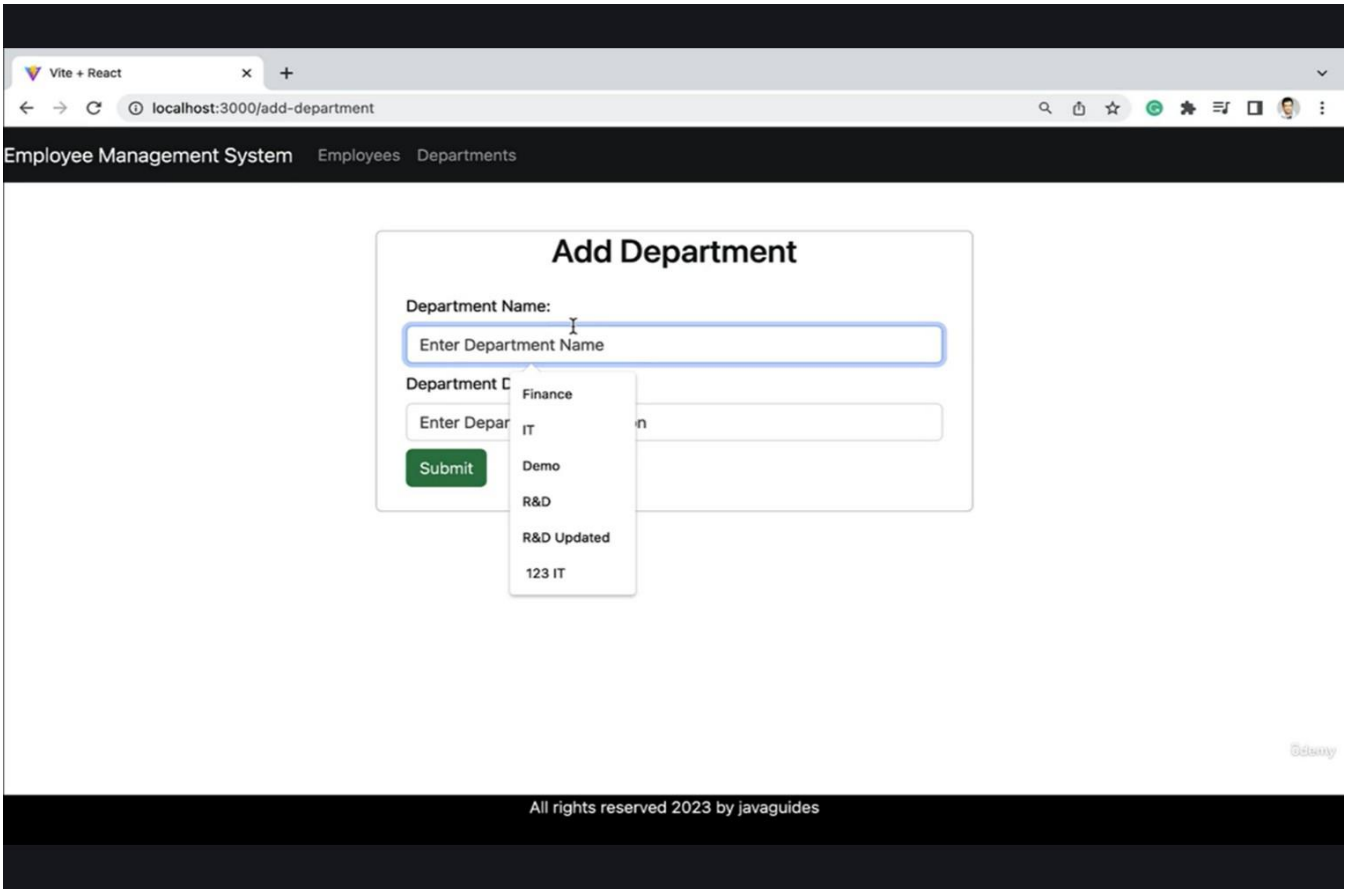


Figure 3.7 Adding department to database

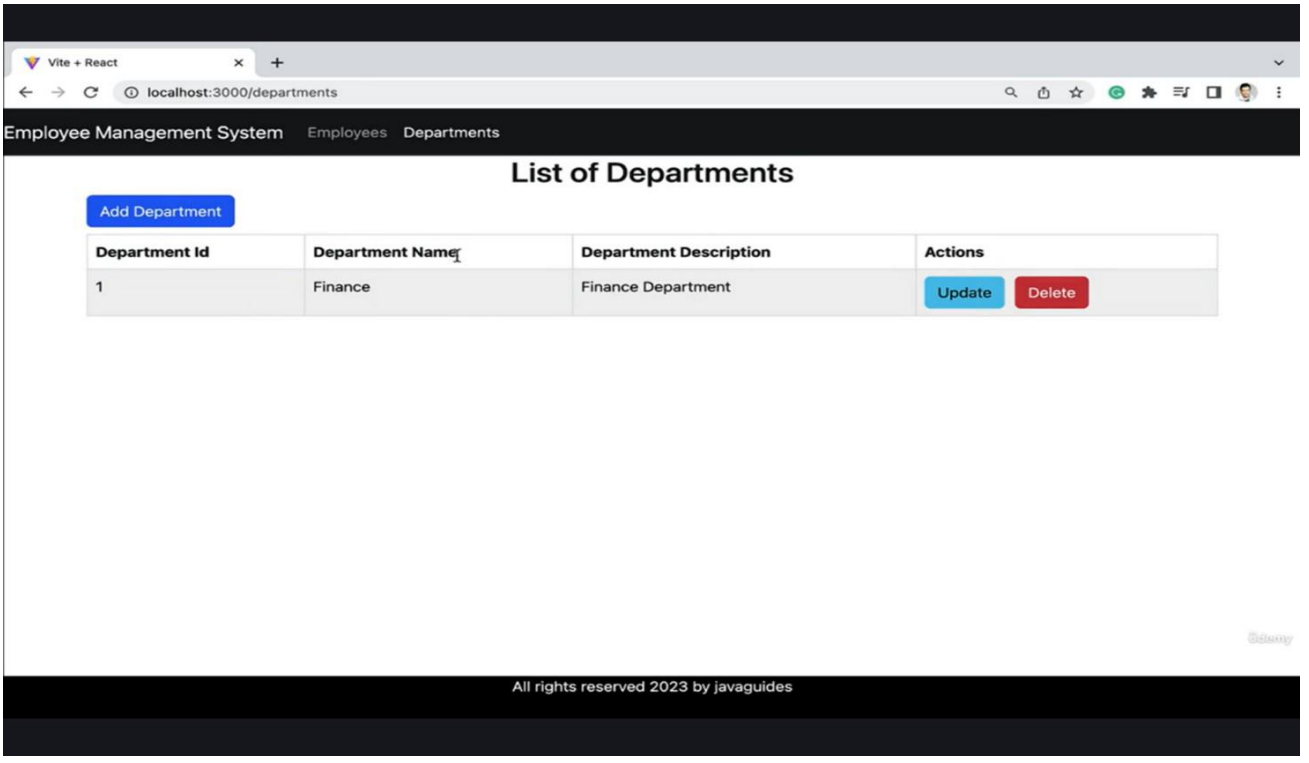


Figure 3.8 Updating the department table

CHAPTER 4:
ACTIVITY LOG

ACTIVITY LOG FOR THE WEEK

weeks	Description of the daily activity	Learning Outcome	Person In-Charge Signature
Week–1	Learned HTML basics: semantic tags, forms, tables, layouts. Practiced CSS styling using Flexbox and Grid.	Ability to design and structure responsive web pages.	
Week-2	Learned JavaScript fundamentals: variables, functions, loops, arrays, and DOM manipulation.	Built foundation in client-side scripting and interactivity.	
Week-3	Developed **Mini Project – EMI Calculator** using HTML, CSS, and JavaScript with validations and dynamic calculations.	Applied frontend skills to build a functional real world mini project.	
Week-4	Studied **SQL Advanced** concepts: joins, subqueries, views, indexes, stored procedures, and constraints.	Gained ability to design and query relational databases efficiently.	

Week-5	Learned **Java fundamentals**: variables, data types, control statements, classes, objects, and OOP principles.	Developed strong programming fundamentals using Java.	
Week-6	Practiced advanced Java: collections, exception handling, file I/O, and multithreading concepts.	Improved problem-solving and application development using Java.	
Week-7	Covered **Data Structures**: arrays, linked lists, stacks, queues, searching, sorting, and recursion.	Built logical thinking and efficient data handling skills.	
Week-8	Learned **Maven**: project structure, dependency management, build lifecycle, and integration with Java projects.	Understood build automation and project management using Maven.	
Week-9	Studied **Spring Framework** fundamentals: Spring Core, dependency injection, annotations, and bean lifecycle.	Gained understanding of backend development and inversion of control.	

Week-10	Worked on **Spring Boot & Spring MVC**: REST controllers, CRUD APIs, validations, and database integration using JPA/Hibernate.	Built scalable backend services using Spring Boot.	
Week-11	Learned **React** fundamentals: JSX, components, props, state, hooks, routing, and API integration.	Developed modern frontend applications using React.	
Week-12	Developed **Major Project – Employee Department Management System** using React frontend and Spring Boot backend. Performed testing and documentation.	Completed an end-to-end full-stack application and gained real-world development experience.	

CHAPTER 5:
CONCLUSION

CONCLUSION

During my internship at **Plasmid Innovation**, I had the opportunity to work extensively in the **Web Development** domain, which significantly enhanced my technical expertise and professional skills. Throughout the internship program, I gained hands-on experience in designing and developing web applications using technologies such as **HTML, CSS, JavaScript, SQL, Java, Data Structures, Maven, Spring Framework, and React**.

Frontend Skills: Designed and developed structured and responsive web pages using HTML and CSS, applying Flexbox, layout techniques, and responsive design principles. Implemented client-side functionality using JavaScript and built a **mini project – EMI Calculator**, focusing on user interaction and dynamic calculations.

- **Java Programming:** Gained a solid understanding of Java fundamentals, including object-oriented programming concepts such as inheritance, encapsulation, abstraction, and polymorphism, along with exception handling and core coding practices.

- **Data Structures:** Studied and implemented fundamental data structures including arrays, linked lists, stacks, queues, searching, and sorting algorithms, which strengthened my logical reasoning and problem-solving skills.

- **SQL Database Management:** Learned advanced SQL concepts such as DDL and DML operations, joins, subqueries, constraints, views, and transactions, enabling effective database design and data manipulation.

- **Build and Dependency Management:** Gained practical knowledge of **Maven**, including project structure, dependency management, and build lifecycle, which helped in managing enterprise-level Java applications.

- **Backend Development:** Worked with the **Spring Framework and Spring Boot** to build backend components, develop RESTful APIs, and implement CRUD operations, understanding the principles of dependency injection and layered architecture.

- **Frontend Framework:** Learned **React**, including components, props, state management, hooks, routing, and API integration, enabling the development of interactive and scalable user interfaces.

- **Project Work:** Contributed to a **major project – Employee Department Management System**, which involved integrating React frontend with Spring Boot backend and database, demonstrating complete end-to-end web application development.

- **Soft Skills:** Improved communication and teamwork skills through regular discussions, presentations, and collaborative project work, gaining exposure to real-world development practices.

Overall, this internship strengthened my foundation in web development, enhanced my ability to work with both frontend and backend technologies, and prepared me to handle real-world software projects with confidence. I am grateful to **Plasmid Innovation** for providing me with this valuable learning experience, guidance, and mentorship.

REFERENCES

1. https://www.w3schools.com/html/
2. https://www.w3schools.com/css/
3. https://spring.io/projects/spring-boot
4. https://hibernate.org/orm/documentation/
5. https://dev.mysql.com/doc/
6. https://www.w3schools.com/
7. https://developer.mozilla.org/
8. https://www.tutorialspoint.com/java/index.htm
9. https://www.geeksforgeeks.org/
10. https://plasmid.teckstac.com/my/